

API Client Platform

(Postman Clone) — SDE Fullstack Assignment

Description

Build a functional clone of the Postman API client that replicates Postman's design, user experience, and core request-building workflows.

The platform should allow users to organize requests into collections, build and send real HTTP requests, inspect responses, manage environments and variables, and experience the clean, developer-focused interface of the original Postman app.

Your implementation should visually and functionally feel like a modern Postman workspace. Unlike a fully mocked app, the request runner should send real HTTP requests and render real responses — this is the core of the experience.

AI Tools Usage

You are allowed and encouraged to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI assistant for development. Use AI as heavily as you like to move fast. However, you must understand every line of code you submit and be prepared to explain your implementation decisions during the evaluation interview.

Technical Stack

- Frontend: Next.js (TypeScript)
- Backend: Python with FastAPI / Django
- Database: SQLite (design your own schema)

Note: the backend should act as a proxy/runner that executes outbound HTTP requests on behalf of the client (to avoid browser CORS limits) and persists collections, history, and environments.

Core Features (Must Have)

1. Workspace Layout & Navigation

Recreate the Postman workspace shell.

- Left sidebar with Collections and History tabs
- Central request builder with tabbed open requests
- Top bar with method selector, URL input, and Send button
- Environment selector

2. Request Builder

Implement a full HTTP request builder.

- Method selection: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS
- URL input with query-param editor (key/value table synced with the URL)
- Headers editor (key/value table)

- Request body editor: none, raw (JSON/text), form-data, x-www-form-urlencoded
- Authorization tab: at minimum None, Bearer Token, and Basic Auth

3. Send Request & Response Viewer

Send real requests through the backend runner and render the response.

- Pretty / Raw response body views with JSON syntax highlighting
- Response status code, time, and size
- Response headers viewer
- Graceful error handling (timeouts, network errors, invalid URLs)

4. Collections (CRUD)

Implement full CRUD for organizing saved requests.

- Create, rename, and delete collections
- Save a request into a collection; edit and delete saved requests
- Nest requests under collections (folders optional)
- All collections and saved requests must persist in the database

5. Environments & Variables

- Create and manage environments with key/value variables
- Reference variables in URL, headers, and body using `{{variable}}` syntax
- Resolve variables at send time based on the selected environment
- Persist environments in the database

6. History

- Automatically record sent requests in a History list
- Re-open a history entry to repopulate the request builder
- Persist history in the database

7. Postman Experience

The application should closely resemble the Postman experience, including:

- Tabs, panes, and resizable layout
- Key/value editor tables
- Forms, modals, search, and filters
- Notifications / toasts
- Settings placeholders

The goal is to make the application feel like Postman rather than a generic form-and-fetch tool.

Mocked / Placeholder Sections

The following can be present as placeholders (a simple “Coming Soon” is sufficient):

- Team workspaces / sharing & collaboration
- Mock servers
- API documentation generation
- Monitors / scheduled runs

- Real user authentication (assume a default logged-in user)

Bonus (Optional)

- Pre-request and test scripts (sandboxed JS execution)
- Import / export collections in Postman Collection v2 JSON format
- Code snippet generation (cURL, fetch, etc.)
- Cookie management
- Dark mode
- Keyboard shortcuts

Important Notes

- UI Design: your application should totally resemble Postman's design. Study Postman's UI carefully before starting.
- Sample Data: seed your database. Seed sample collections, a couple of environments, and some history pointing at a public test API (e.g. a JSON placeholder or httpbin-style endpoint) so the app is immediately usable.
- Database Design: design your own database schema. This will be evaluated.
- README File: include setup instructions, tech stack used, architecture overview, database schema, and any assumptions made.
- Original Work: plagiarism from existing repositories will result in immediate disqualification.

Deliverables

- Source Code: a public GitHub repository containing frontend/ and backend/.
- Documentation: a README with setup instructions, architecture overview, database schema, and API overview.
- Demo: a hosted, working link.

Submission

- Upload your code to GitHub and ensure the repository is public.
- Deploy your application (Vercel, Netlify, Render, Railway, or any cloud service).
- Submit both the GitHub repository link and the deployed application link.

Evaluation Criteria

Criteria	What We Look For
Functionality	All core features working correctly, including sending real requests and rendering responses
UI/UX	Visual similarity to the original app's design and UX patterns
Database Design	Well-structured schema with proper relationships

Backend / API Design	Clean, sensible API design and architecture
Code Quality	Clean, readable, and well-organized code
Code Modularity	Proper separation of concerns, reusable components
Code Understanding	Ability to explain your code during evaluation

Timeline

Estimated effort: approximately 24 hours of work.

Submission Deadline: as communicated alongside this assignment.

Good luck!